

Performance Testing

Date	10 July 2026
Team ID	xxxxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Step 1: Testing Overview

Testing Tool Used	Postman (API load), Apache JMeter (concurrent request simulation)
Type of Testing	Load Testing, Response-Time Testing
Target Module / API	/predict and /api/predict endpoints of the Flask fraud-scoring app
Test Environment	Local (Flask dev server) + Ngrok-tunnelled public URL
Test Date	04 July 2026

Step 2: Test Scenarios

S.No	Test Scenario	Virtual Users	Duration (sec)	Expected Outcome
1	Single transaction prediction under normal load	10	30	All requests return < 2s with correct JSON response
2	Burst of simultaneous single predictions	100	30	No dropped requests; response time may rise but stay reasonable
3	Batch CSV upload (1,000 rows)	5	60	Batch completes without server error
4	Sustained load	50	300	Stable memory/CPU, no crash over 5 minutes

Step 3: Performance Test Results

S.No	Metric	Target	Actual	Status	Remarks
1	Response Time (Avg)	< 2 sec	~0.4 sec	Pass	Single prediction is fast; model inference is lightweight
2	Response Time (Max)	< 5 sec	~1.8 sec	Pass	Observed under 100 concurrent virtual users
3	Throughput (Req/sec)	-	~45 req/sec	Pass	On local dev server; would scale further under IBM Watson ML
4	Error Rate	< 1%	0%	Pass	No failed requests during test window
5	CPU Utilization	< 80%	~55%	Pass	Local machine, single Flask worker

6	Memory Utilization	< 80%	~40%	Pass	Model + scaler are small (few MB)
---	--------------------	-------	------	------	-----------------------------------

Step 4: Observations & Analysis — The Flask app comfortably meets response-time targets for single-transaction scoring, which is the primary real-world use case (scoring one transaction as it happens). Throughput on a single local dev-server process is modest; for production-scale traffic, deploying via IBM Watson Machine Learning (or running Flask behind Gunicorn with multiple workers) would be needed to handle higher concurrent volume.